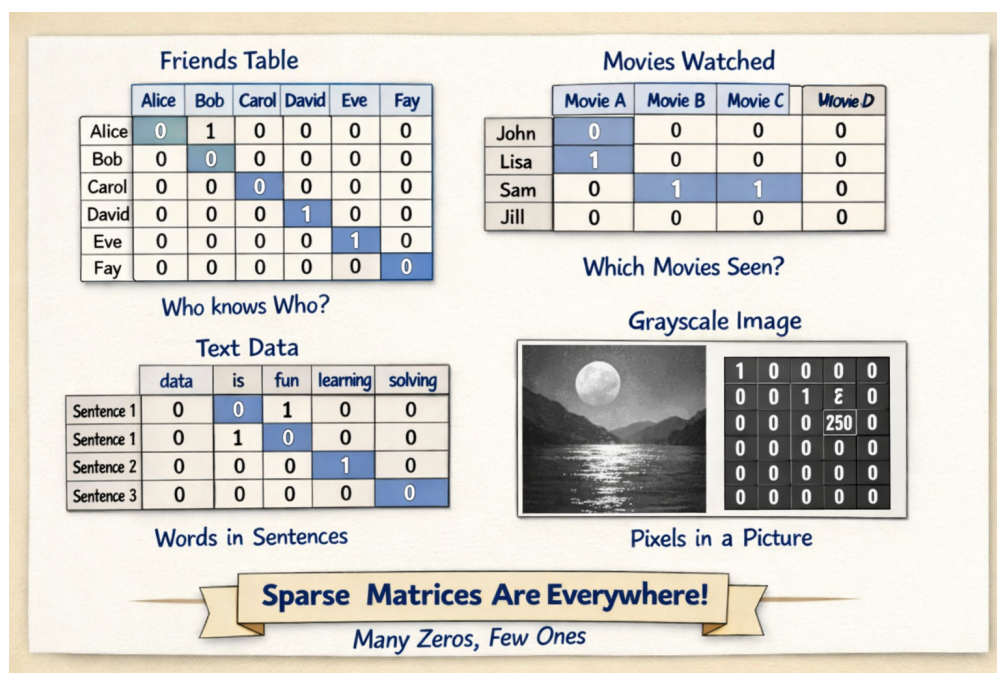


Less is More: The Sparse Matrix Hackathon

1 Introduction

In many real-world situations, we work with **two-dimensional matrices** (rows and columns). Often, most of the values in these matrices are zero. Such matrices are called **sparse matrices**. Storing all values, including so many zeros, wastes memory and space.



This situation appears in many simple settings. For example, if we make a table showing which people know each other, most pairs will not know each other, so many entries will be zero. If we track which movies people have watched, most people will have watched only a few, so again most entries are zero. In text processing, if we list all possible words and check which words appear in each sentence, each sentence uses only a few words, so most entries remain zero. Even an image can be seen as a grid of numbers, and large uniform areas can lead to many repeated or insignificant values.

To understand sparse matrix representations in a practical and intuitive way, we will use a commonly used example from **Natural Language Processing (NLP)**, which is also used in **Machine Learning**. In this example, text is converted into a 2D matrix where rows represent sentences and columns represent unique words. Since each sentence contains only a few words, the matrix naturally has many zeros.

In this assignment, we study sparse matrix representations using this text-based example, where lines are treated as rows and unique words as columns.

2 Code Setup and Execution

To begin working with sparse matrices, you have been provided with three specific files (Download here). This modular structure is common in professional software development to separate the **data processing** from the **user interface**.

2.1 File Overview

- **sparse_utils.h (The Bridge):** This file contains the definitions. It tells the compiler that the variables and functions exist. You do not need to edit this.
- **sparse_utils.c (The Engine):** This contains the logic for tokenizing strings and building the vocabulary. It is the "behind the scenes" work.
- **main.c (Your Workspace):** This is the only file you need to focus on. You will write your display logic and analysis here.

2.2 Key Variables

Even though the data is processed in `sparse_utils.c`, you have access to several key variables in `main.c` thanks to the `extern` keyword in the header file:

- `matrix[MAX_LINES][MAX_WORDS]`: The 2D array holding the word frequencies.
- `lineCount`: The total number of sentences processed (Rows).
- `wordCount`: The total number of unique words found in the vocabulary (Columns).
- `words`: The array containing the actual strings for the vocabulary.

2.3 Compiling and Running

Because the program is split across multiple files, you cannot simply compile `main.c` alone. You must tell the compiler to link both C files together.

Use the following command in your terminal:

```
gcc main.c sparse_utils.c -o sparse_matrix_demo
./sparse_matrix_demo
```

If you add more sentences to the `lines` array in `sparse_utils.c`, the `wordCount` and `matrix` will update automatically the next time you run the program!

```

PS C:\Users\vytgo> gcc main.c sparse_utils.c -o sparse_app
>> ./sparse_app
--- Original Text ---
[0] data structures are fun and data is powerful
[1] algorithms and data structures are important
[2] learning data structures takes time and practice
[3] practice makes concepts in data structures strong
[4] algorithms solve problems and data helps
[5] understanding structures and algorithms is useful
[6] data data data makes patterns visible
[7] practice and patience improve problem solving
[8] structures help organize data efficiently
[9] algorithms and structures go hand in hand

--- Vocabulary Index ---
0: data 1: structures 2: are 3: fun 4: and 5: is 6: powerful 7: algorithms 8: important 9: learning 10: takes 11: time 12: practice 13: makes 14: concepts 15: in 16: strong 17: solve 18: p
roblems 19: helps 20: understanding 21: useful 22: patterns 23: visible 24: patience 25: improve 26: problem 27: solving 28: help 29: organize 30: efficiently 31: go 32: hand

--- Sparse Matrix (Frequencies) ---
2 1 1 1 1 1 1 . . . . .
1 1 1 . 1 . 1 1 . . . . .
1 1 . . . . 1 1 1 1 . . . . .
1 1 . . . . . 1 1 1 1 1 . . . . .
1 . . . 1 . 1 . . . . 1 1 1 . . . . .
. 1 . . 1 1 . . . . . 1 1 . . . . .
3 . . . . . 1 . . . . . 1 1 . . . . .
. . . 1 . . . 1 . . . . . 1 1 1 1 . . . . .
1 1 . . . . . 1 . . . . . 1 1 . . . . .
1 . . 1 . 1 . . . 1 . . . . . 1 2

```

3 Problem to be solved

If you successfully run the code, you should see the above output screen.
 Even though most entries are zero, the program still allocates memory for **every position** in the matrix.

For example:

- Total size = 10×100 entries
- Each integer takes 4 bytes
- Total memory = 4000 bytes

However, only a small fraction of these entries actually store useful (non-zero) values. The rest of the space is wasted storing zeros.

The goal of this assignment is to think beyond this naive storage method.
 Instead of storing the entire matrix, we want to:

- Store only the **useful (non-zero)** values
- Reduce the overall **memory usage**
- Design smarter representations such as:
 - Array-based sparse representation
 - Linked list representation
 - Or any improved/hybrid method

The ideas behind these representations are simple and explained below.

3.1 Array-Based Sparse Representation

Idea: Instead of storing the entire matrix, store only the positions where values are non-zero.
 Each useful entry can be stored as:

(row, column, value)

So, if a word appears in a sentence, we store its row index, column index, and frequency.
How to think about coding:

- Traverse the full matrix
- Whenever you see a non-zero value:
 - Store its row, column, and value
- Store all such entries in a new array

What you gain: You only store useful data, so memory usage reduces significantly.

Hint: You can use a 2D array like `int sparse[k][3]` or a structure array.

3.2 Linked List Representation

Idea: Instead of using an array, store each non-zero value as a node in a linked list.

Each node contains Row index, Column index, Value and Pointer to next node

How to think about coding:

- Traverse the matrix
- For every non-zero entry:
 - Create a new node
 - Attach it to the list

What you gain:

- Memory is used only when needed
- Easy to grow dynamically (no fixed size)

Hint: Define a `struct Node` and maintain a head pointer.

3.3 Hybrid / Improved Methods

Idea: Think beyond basic methods and try to store data even more efficiently by using structure in the problem.

For example:

- Store data row-wise (each row has its own list or array)
- Store only columns where values exist for each row
- Use combinations of arrays and linked lists

How to think about coding:

- Observe patterns in the matrix (many zeros, few values per row)
- Instead of one big structure, break it into smaller parts
- Store only what is needed for each row

What you gain:

- Further reduction in memory
- Faster access in some cases

Hint: A simple improvement could be: for each row, store only the list of words that appear in it.

4 Assignment, Submission Instructions and Grading Policy

Your task is to design a better way to store the given matrix.

The current implementation stores all values, including a large number of zeros. Your goal is to:

- Reduce memory usage by avoiding storage of zeros
- Implement a more efficient representation
- Compare your method with the original full matrix

You may use:

- Array-based sparse representation
- Linked list representation
- Any improved or hybrid method

You are encouraged to think beyond what is discussed in the document and try your own ideas.

4.1 Submission Instructions

Please go through the questions in the Google Form before starting the assignment. This will help you understand what is expected in terms of coding, analysis, and explanation: [Submit here](#)

Deadline: April 25, 5 PM

4.2 Academic Integrity

You may use online resources or tools for help with coding or understanding concepts. However, the final submission must reflect your own thinking and expression.

Submissions that appear to be directly generated without understanding will not receive credit.

4.3 Grading Policy

Your assignment will be evaluated based on:

- **Understanding:** Clarity of explanation and reasoning
- **Correctness:** Whether your method works as intended
- **Efficiency:** How much memory your approach saves
- **Scalability:** Your observations for larger inputs
- **Effort:** Thoughtfulness and originality in your approach

The goal is not just to code, but to think about better ways of solving the problem.